



MECHATRONICS SYSTEM DESIGN

3D Scanner System

By

Ibrahim Ezzat

Mohamed Amr

Adham Amr

Maryam Wael

Mohamed Hisham

Sama Atef

Supervised By

Prof. Dr. Abdelhalim Bassiony

Dr. Mohamed Adel

2025

Table of contents

1. Abstract
 - a. Brief overview of the 3D scanner project
 - b. Key objectives and outcomes
2. Introduction
 - a. Problem Statement
 - b. Importance and Applications of 3D Scanning
 - c. Project Scope
 - d. Objectives
3. Literature Review
 - a. Types of 3D Scanners
 - b. Comparison and Gap Analysis
4. System Overview
 - a. General process flow diagram
5. Mechanical Design
 - a. Scanner Frame and Structure
 - b. Rotating Platform
 - c. Manufacturing selection
 - d. Assembly
6. Electrical System
 - a. Microcontroller
 - b. Sensor selection
 - c. Working Principle of the sensor
 - d. Motors & Drivers
 - e. Circuit schematic
7. Software System
 - a. Motor sequence test
 - b. Sensor test
 - c. Data Transmission & Storage,
 - d. Meshlab
8. Modeling
 - a. Stepper motor modeling
 - b. Stepper motor in simulink
 - c. Motor driver modeling
 - d. Fully integrated model
 - e. Motor response

Abstract

This project presents the design, fabrication, and modeling of a low-cost, Arduino-based 3D scanner tailored for small-scale prototyping and reverse engineering applications. In contrast to commercial 3D scanners that rely on expensive technologies such as time-of-flight, triangulation, or structured light, the proposed system offers a modular and budget-friendly alternative by employing an infrared (IR) distance sensor.

The mechanical design incorporates a custom-built lifting and rotating platform, enabling the scanner to capture object geometry from multiple angles. The electrical system is built around an Arduino Uno microcontroller, interfaced with an IR sensor, LCD display, SD card module, and stepper motors to ensure accurate data acquisition and real-time feedback.

Scanned data is stored as raw distance readings, which are later converted into 3D point cloud data and reconstructed using MeshLab. Additionally, the scanning mechanism, including motor control and sensor feedback, is modeled and simulated using MATLAB/Simulink, allowing for system-level validation and performance optimization through control system tuning.

By leveraging cost-effective hardware and open-source software tools, this scanner provides an accessible and educational platform for hobbyists, students, and educators, bridging the gap between affordability and functionality in the field of 3D scanning technology.

Introduction

3D scanning is the process of digitally capturing the geometry of a physical object, environment, or person to create an accurate three-dimensional representation. Unlike standard cameras that record only 2D images, 3D scanners capture width, height, depth, and sometimes even surface texture or color, generating a dense point cloud composed of millions of spatial data points.

Modern industries increasingly rely on 3D scanning for applications in quality control, reverse engineering, rapid prototyping, and cultural preservation. Among the various techniques available, non-contact laser and infrared scanning methods are widely favored for their ability to obtain high-precision measurements without altering or damaging delicate surfaces.

However, commercial 3D scanning systems remain cost-prohibitive for hobbyists, students, and small-scale prototyping applications. These systems often rely on expensive components such as structured light projectors or time-of-flight sensors, presenting a barrier to wider adoption in educational and maker environments.

To address this gap, this project proposes the development of a low-cost, Arduino-based 3D scanner using an infrared (IR) distance sensor, controlled via a custom-built electromechanical system. The scanner facilitates automated point cloud acquisition, enabling users to perform reverse engineering—capturing the shape of physical objects, constructing digital 3D models, and then modifying or replicating them through software tools.

This introduction sets the stage for a deeper exploration of the system's mechanical design, electronic architecture, data acquisition software, and MATLAB/Simulink-based simulation, all aimed at offering an affordable, modular, and educational 3D scanning solution.

Literature Review

3D scanners are broadly classified into contact and non-contact systems. Contact-based scanners, such as the Coordinate Measuring Machine (CMM), use mechanical probes to touch the surface of an object and determine precise coordinates along the X, Y, and Z axes. Despite their high accuracy, CMMs are slow, expensive, and unsuitable for fragile or soft materials due to the risk of surface deformation or damage.

To overcome the limitations of contact-based systems, various non-contact scanning techniques have emerged such as:

- **Laser triangulation** uses reflected laser beams and a positional detector to measure distance based on the angle of reflection.
- **Structured light systems**, introduced by Strauss in 1992, project coded light patterns (stripes, grids, or dots) onto an object and use stereo cameras to calculate depth via triangulation.
- **Time-of-flight (ToF) scanners**, made possible after the invention of the laser in the 1960s, measure the time it takes for light to travel to the object and reflect back, offering long-range but often less accurate data.

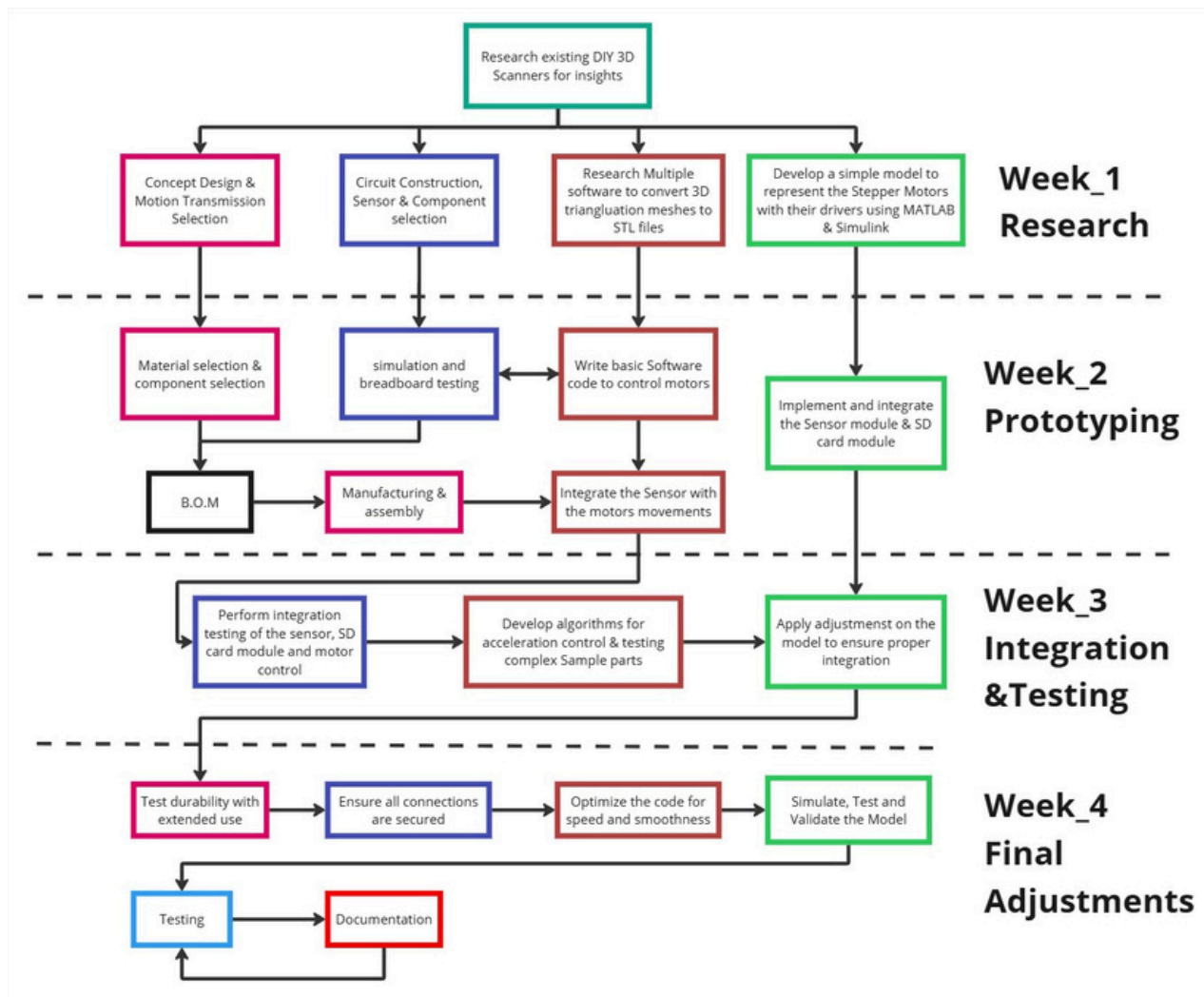
Although commercial 3D scanners deliver high-speed and accurate results, their high cost, complexity, and proprietary software make them inaccessible for students, hobbyists, and low-budget research applications. These systems often require high-performance processing units and well-controlled lighting conditions, which can be a barrier in educational or DIY environments.

To address these limitations, this project proposes a cost-effective, Arduino-based 3D scanning system. The scanner employs:

- A VL6180X Time-of-Flight (ToF) IR sensor that operates on the principle of triangulation and is suitable for short-range precision scanning.
- NEMA17 stepper motors to enable Z-axis linear motion and rotational movement for full 360° scanning.
- A modular control unit built around an Arduino Uno, with user interaction via LCD, and data logging via SD card.

The acquired point cloud data is later processed using MeshLab to reconstruct a 3D mesh, enabling reverse engineering, educational prototyping, and model customization at a fraction of the cost of commercial alternatives.

System Overview



Figa.1 General process flow diagram

This diagram outlines a 4-week workflow for designing and building a DIY 3D scanner, divided into research, prototyping, integration/testing, and final adjustments. Key steps include:

- **Research:** Investigating existing DIY 3D scanners, software for mesh conversion (e.g., STL files), and component selection.
- **Prototyping:** Designing the system, simulating motor control (using MATLAB/Simulink), breadboard testing, and writing basic motor control code.
- **Integration & Testing:** Combining sensors, SD card modules, and motors, followed by algorithm development for acceleration control and durability testing.
- **Final Adjustments:** Optimizing code, securing connections, validating the model, and documenting the process.

The process emphasizes iterative testing, simulation, and integration to ensure functionality and durability.

System Overview

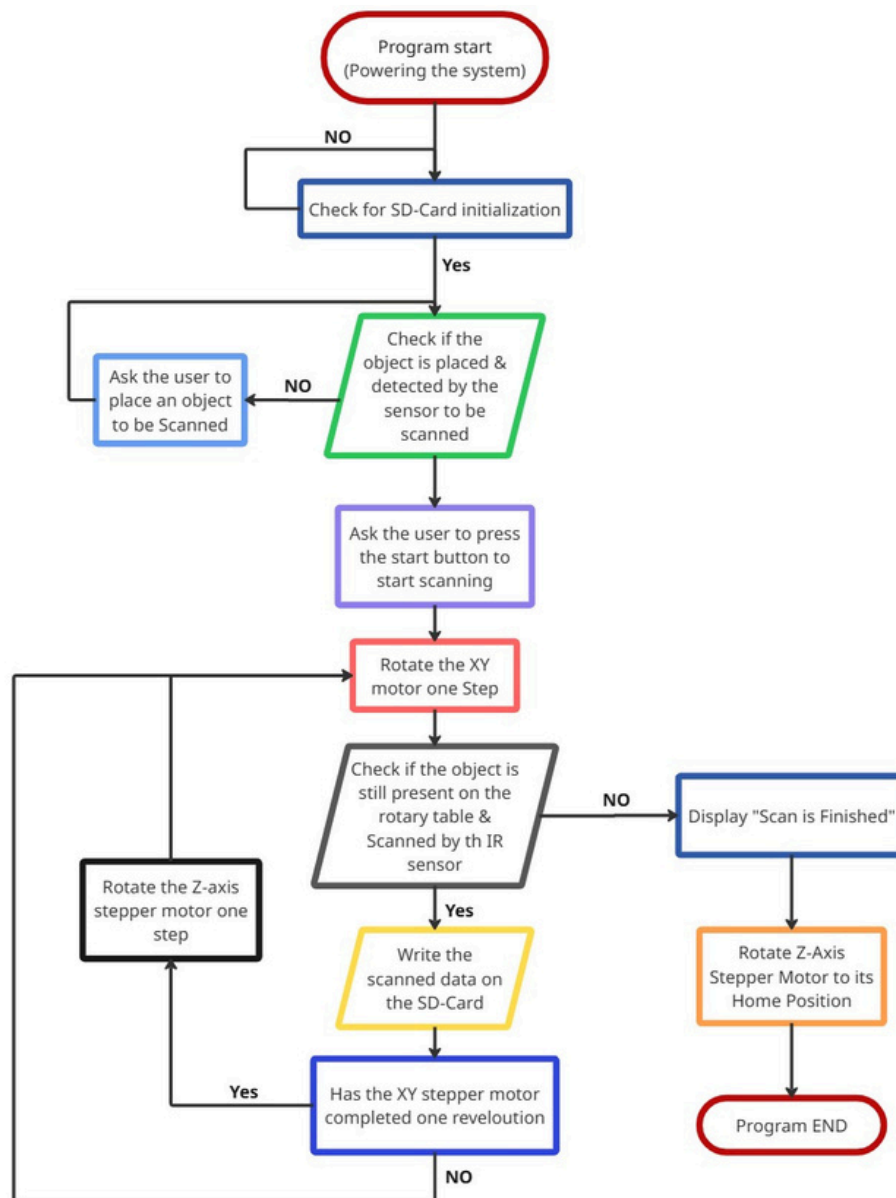


Fig.2 System's flowchart

This diagram shows the step-by-step operational flow of a DIY 3D scanner system. The process begins with powering on the system and checking if an SD card is initialized for data storage. If not, the system halts. Next, it checks if an object is placed and detected by the sensor. If no object is found, the user is prompted to place one and press a start button to begin scanning.

Once scanning starts, the XY stepper motor rotates the object one step at a time while the Z-axis motor adjusts vertically, ensuring full coverage. The system continuously checks whether the object is still present and being scanned. Data from the scan is written to the SD card after each step. The loop repeats until the XY motor completes one full revolution.

Mechanical Design

Scanner Frame and Structure

The overall structure of the 3D scanner is designed with simplicity and rigidity in mind to ensure accurate and stable scanning.

The main frame consists of a vertical Z-axis assembly, supported by two smooth guide rods and a centrally placed lead screw for linear motion. The use of linear bearings along the guide rods reduces friction and ensures a stable and vibration-free translation of the sensor head. The structural components are mounted on a flat baseplate, providing a stable reference surface and reducing misalignment during operation. The IR sensor is placed on the mounting Plate supported by the two Stainless steel rods. The actuator of the linear Z-axis mechanism is simply a nema

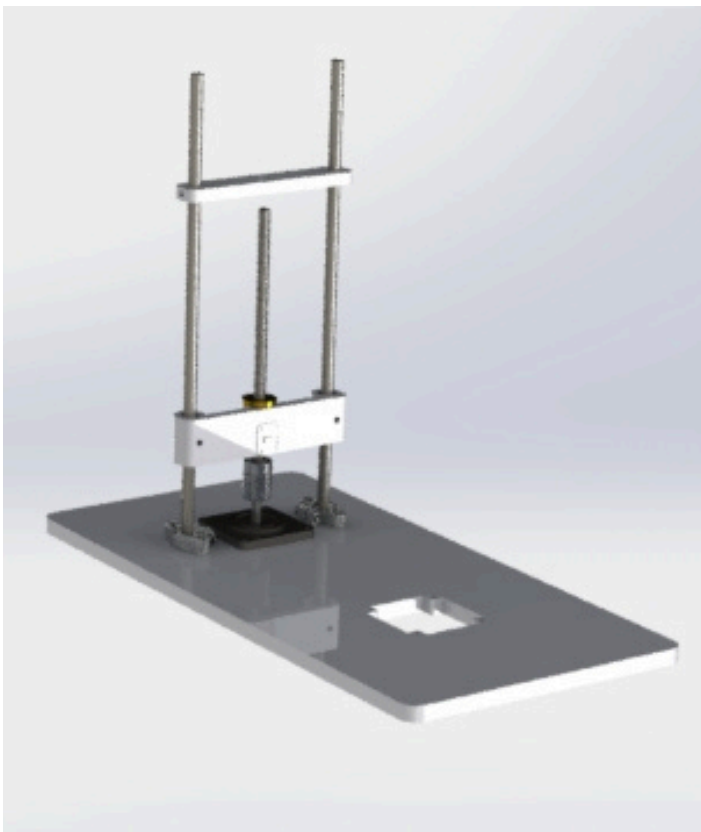


Fig b.1 Frame and structure



Fig b.2 Frame & structure Front_View

Rotating Platform

The rotating platform, which serves as the object stage, is placed at the opposite end of the baseplate. It consists of a circular disk mounted on a NEMA23 stepper motor, allowing precise rotational control of the object to be scanned. This enables 360° coverage as the sensor captures measurements from various angles. The disk is dimensioned to accommodate small- to medium-sized objects, and the direct motor mounting ensures repeatable and synchronized rotations

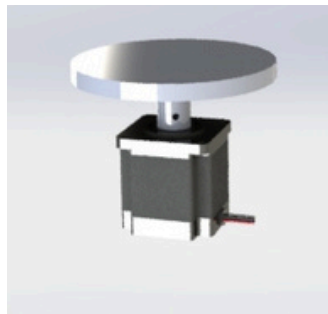


Fig b.3 Rotating Platform

Manufacturing selection

The mechanical components of the 3D scanner were fabricated using a combination of laser cutting and Fused Deposition Modeling (FDM) 3D printing, chosen for their accessibility, precision, and cost-effectiveness.

The base plate of the scanner, which serves as the foundation for the rotating platform and vertical guide system, was manufactured using laser cutting technology. The material selected was Wood

All non-flat components, including the sensor holder, motor mounts, lead screw bracket, and platform disk, were fabricated using 3D printing (FDM). These parts were designed in CAD and printed using PLA

Assembly

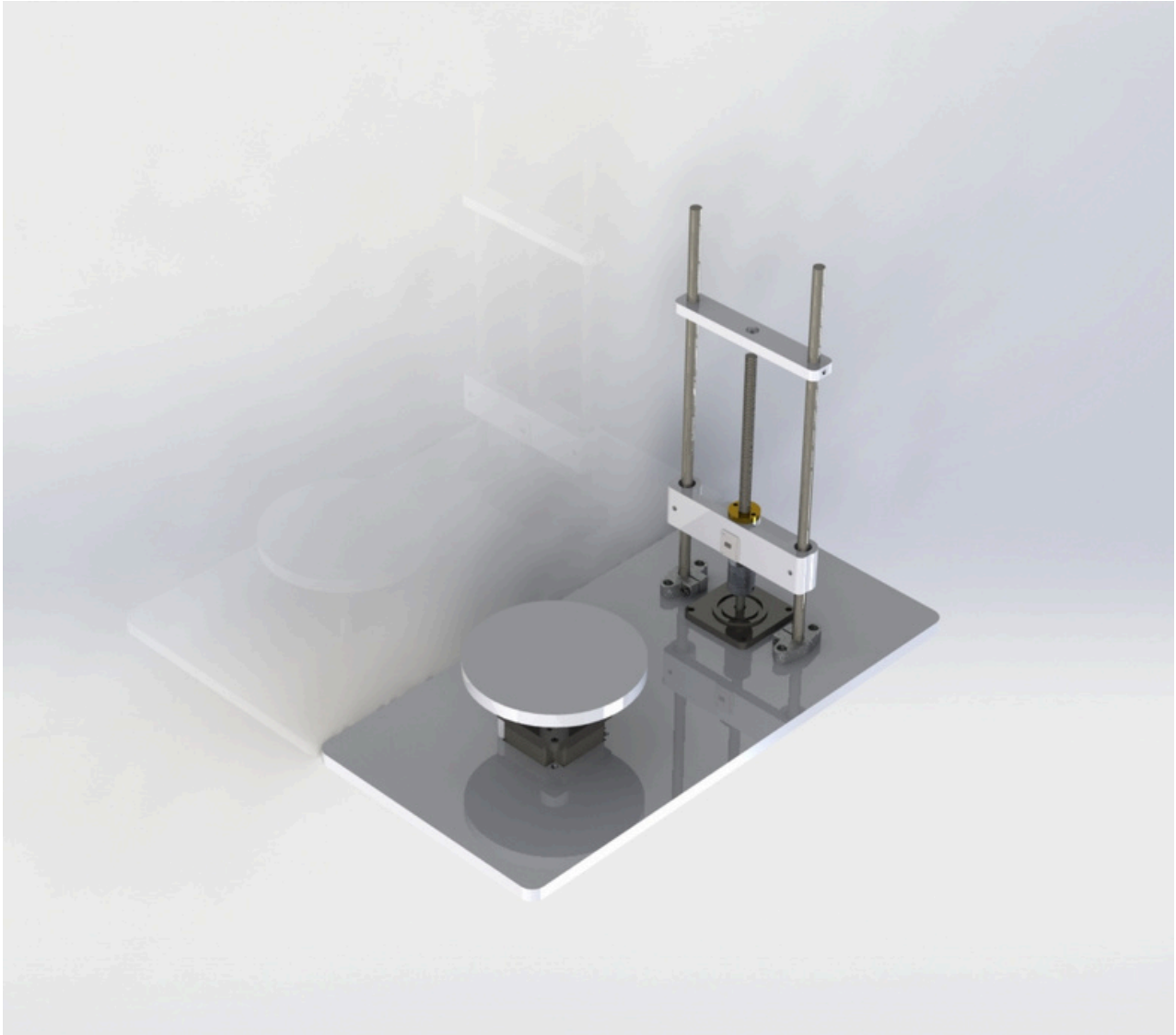


Fig c.1 Final Assembly

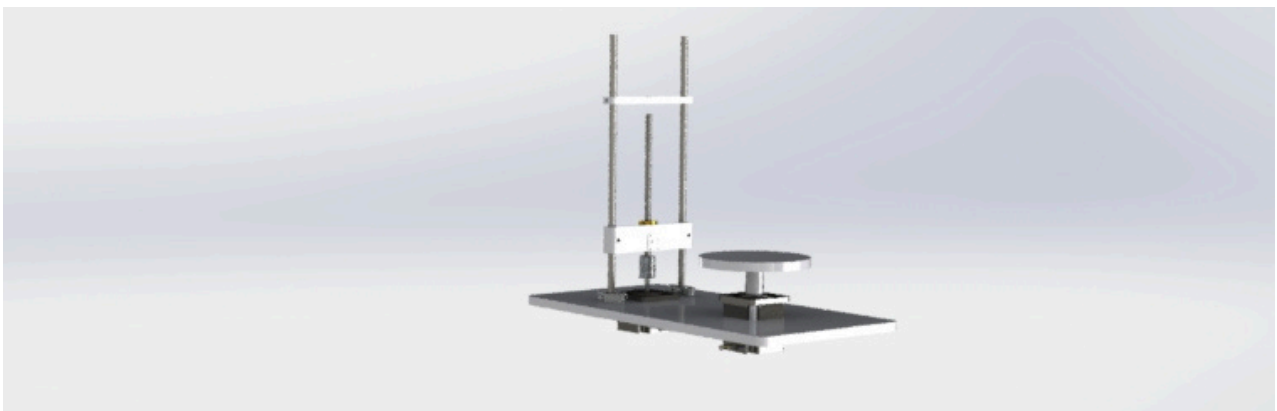


Fig c.2 Final 3D scanner Model

Electrical System

Microcontroller

The system is controlled by a microcontroller Arduino uno coordinates all operations. Its primary tasks include:

- Driving the stepper motors via motor drivers.
- Reading data from the selected sensor (laser or ultrasonic).
- Managing user inputs (start/stop buttons) and displaying instructions via the 16x2 LCD.
- Logging scanned data to the SD card for later conversion into STL files.
- The firmware must ensure real-time synchronization between motor movements and sensor readings to achieve accurate scanning.

Sensor Selection

After evaluating multiple sensors, the TOF050C Module VL6180X laser sensor was chosen for its balance of precision ($\pm 1\text{mm}$ at short range) and speed (30–100ms per reading). Key considerations included:

- Range & Resolution: The laser sensor provides finer detail (1mm resolution) compared to the ultrasonic alternative ($\pm 10\text{mm}$ error).
- Limitations: Performance degrades at long distances ($>30\text{cm}$) and under bright ambient light.
- Alternative options like the ultrasonic sensor (HY-SRF05) were considered for cost savings but rejected due to lower accuracy and wider beam angle, which could miss fine details.

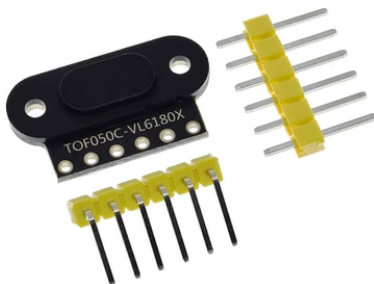


Fig d.1 TOF Sensor

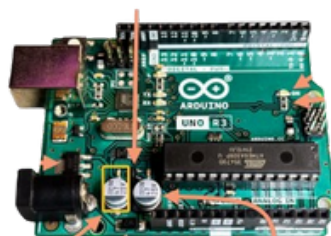


Fig d.2 arduino uno

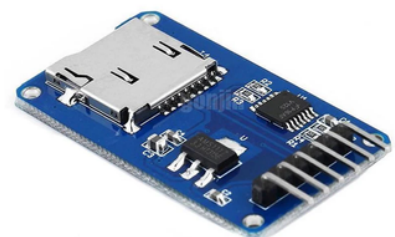


Fig d.3 SD card module

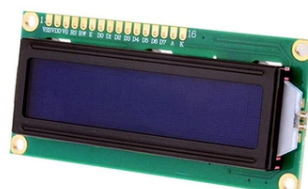


Fig d.3 arduino uno

Working Principle of the sensor

The VL6180X operates by sending out a laser light plus from the emitter, receiving the reflected light from an object in the detector, and based of the time it took (time-of-flight) then computes the distance to the object. The picture below shows the emitter (illumination) and detector (view) cones.

Data Collection from the VL6180X Laser Sensor for 3D Mapping

Each Data Point Includes:

- Distance (r): distance from the center to the object's surface.
- Horizontal Angle (θ): Rotation angle of the platform or base.

3D conversion:

- $X = r \times \cos(\theta)$, $Y = r \times \sin(\theta)$
- $Z = r$

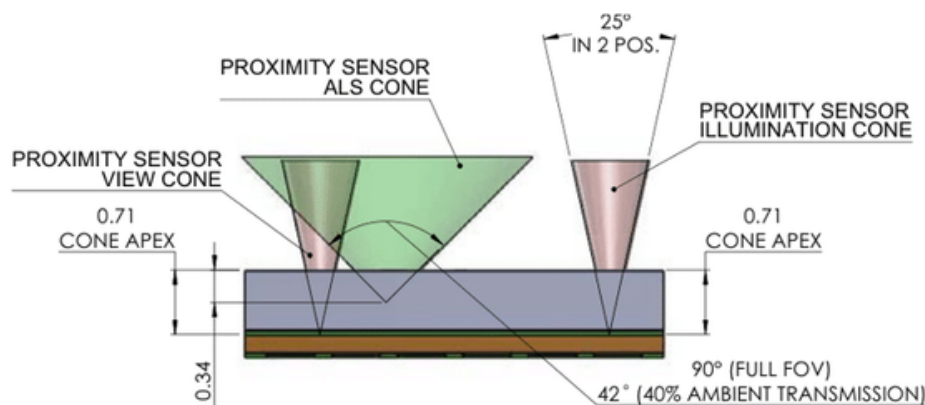


Fig d.4 Working Principle of TOF sensor

Motors & Drivers

- Stepper Motors: Two NEMA 23 stepper motors control the XY rotation (for object positioning) and Z-axis adjustment (for height variation). These motors provide sufficient torque and precision for small-scale scanning.
- Motor Drivers: A4988 drivers enable microstepping, allowing smooth and precise incremental movements. The firmware includes acceleration/deceleration algorithms to minimize vibration and ensure consistent scanning speed.

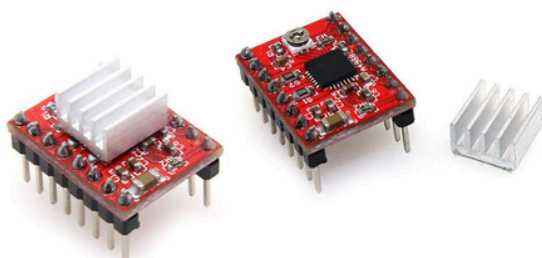


Fig d.5 A4988 Motor Driver

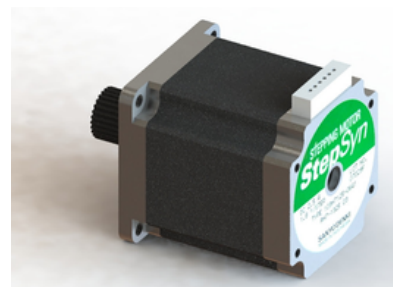


Fig d.6 Nema 23 Stepper motor

Circuit schematic

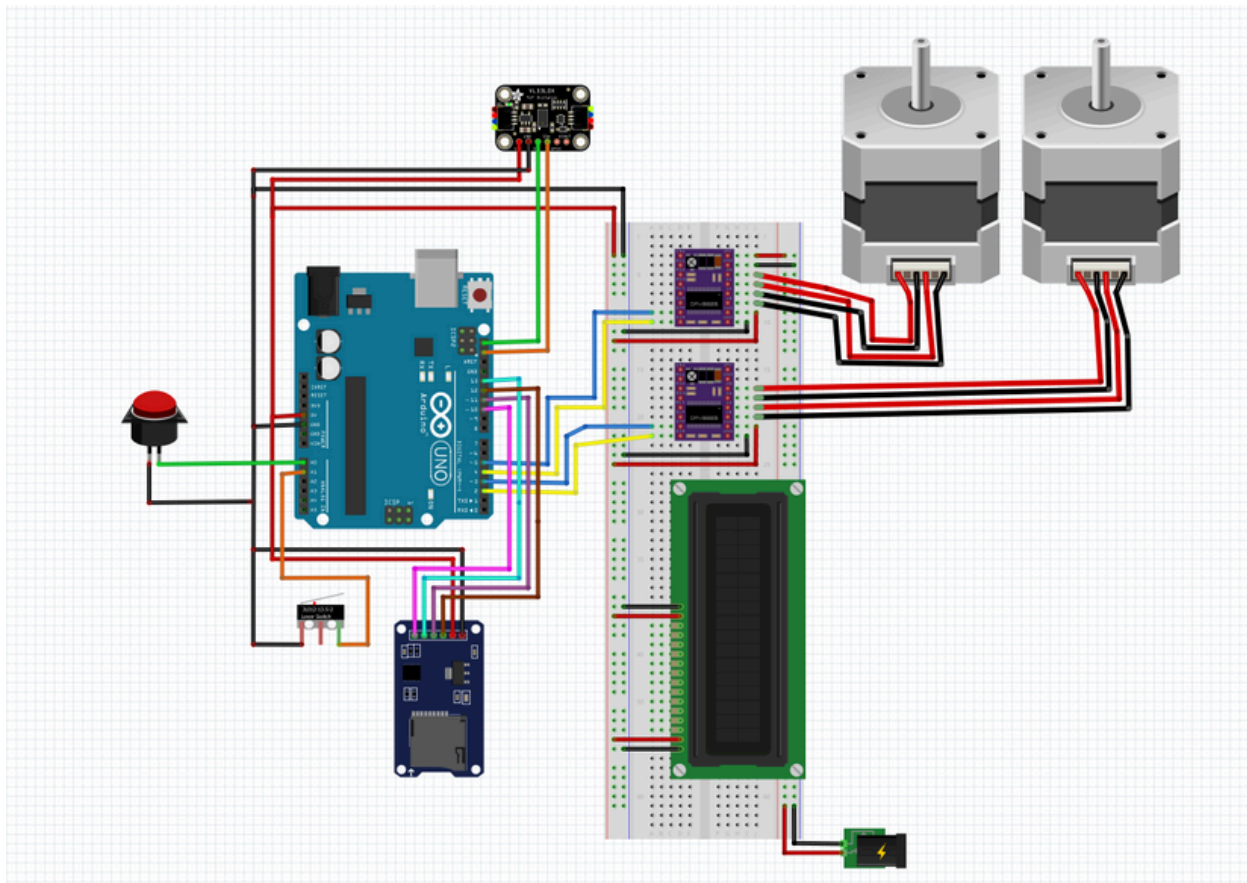


Fig d.7 Schematic on Fritzing

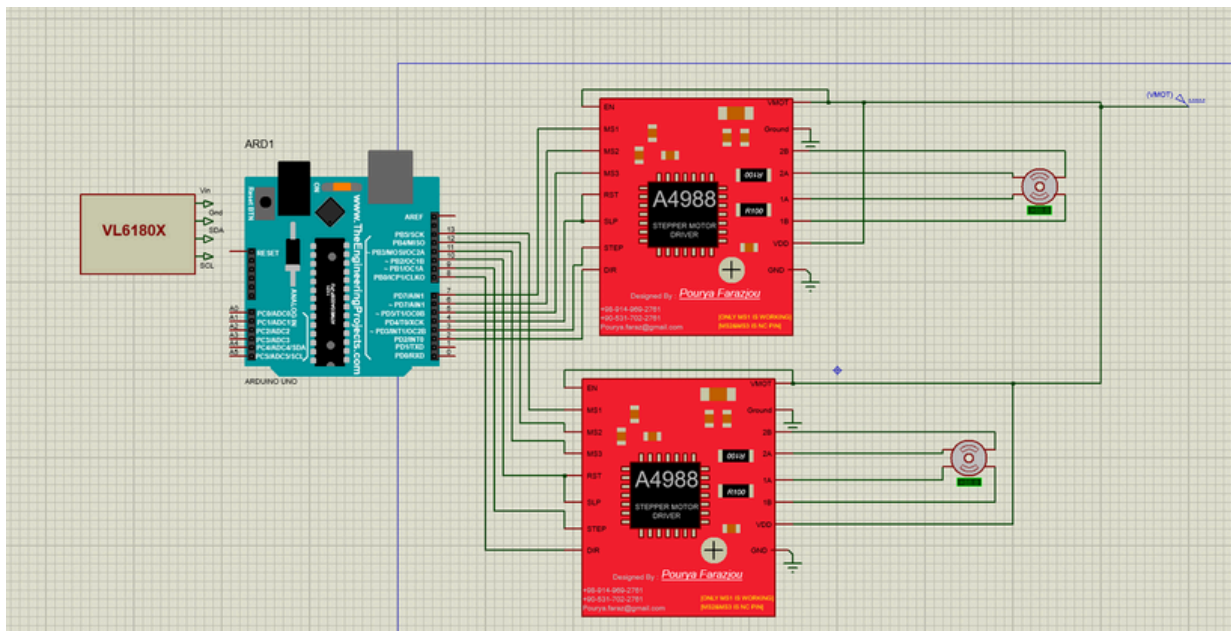


Fig d.7 Schematic on Proteus

Software System

Motor Sequence test

Before proceeding with the full integration of the scanning system, a motor sequence test was conducted to validate the correct operation of both stepper motors. The objective was to ensure:

- The Z-axis motor (responsible for rotating the object) moves in precise and controlled increments.
- The Y-axis motor (responsible for moving the sensor vertically) accurately reaches the desired positions.

The test involved running each motor independently and then in combination to simulate the coordinated scanning motion. During this process, the following parameters were verified:

- Step accuracy and repeatability
- Motor speed and acceleration response
- Synchronization between both axes for smooth, continuous scanning

This sequence test proved essential in identifying and resolving potential issues such as missed steps, incorrect motor direction wiring, or control loop delays. After successful validation, the system was deemed ready for the full integration of the scanning and data acquisition process.

Sensor Test

Prior to integrating the VL6180X time-of-flight distance sensor into the scanning system, a standalone test was conducted to evaluate its functionality, accuracy, and stability. The sensor communicates via the I²C protocol, allowing for efficient and reliable data transmission with the microcontroller.

Objectives of the Test:

- Verify that the sensor accurately measures distances.
- Assess the consistency and stability of the distance readings over time.
- Determine the sensor's effective range and response time under varying conditions.

Test Procedure: The VL6180X sensor was connected to a microcontroller, and a basic script was developed to continuously read and display real-time distance measurements. The test involved:

- Placing various objects at known distances along the X-axis to validate measurement accuracy.
- Repeating the test in different ambient lighting conditions to evaluate sensitivity to environmental factors.
- Using objects with different surface materials to assess the sensor's reliability and performance across reflective and non-reflective surfaces.

The results confirmed that the VL6180X sensor provides stable and accurate readings within its effective range, making it suitable for integration into the 3D scanning system.

Data Transmission & Storage

The scanning system supports two modes of operation for data acquisition and storage, both aimed at capturing accurate spatial data for 3D reconstruction.

1. Real-Time Data Transmission via USB

The microcontroller can be directly connected to a PC through a USB interface, enabling real-time data transmission over a serial connection. During the scanning process, the system continuously sends:

- Distance readings from the VL6180X sensor
- Corresponding motor positions on the Y-axis (sensor height) and Z-axis (object rotation)

A custom Python script on the PC receives and processes the serial data, converting it into a structured .xyz file format, which is compatible with 3D visualization software such as MeshLab.

2. Offline Data Logging to SD Card for portable or offline operation, the system can store data locally using an SD card module. In this mode:

- The microcontroller logs the sensor measurements and motor positions into a file (e.g., scan_data.txt) on the SD card during scanning.
- After completion, the SD card can be removed and inserted into a computer for post-processing.

This flexibility ensures the system is adaptable for both lab-based and field applications.

Meshlab

Once the scan data is collected, the final step involves visualizing the resulting 3D point cloud using MeshLab, an open-source tool for 3D mesh processing.

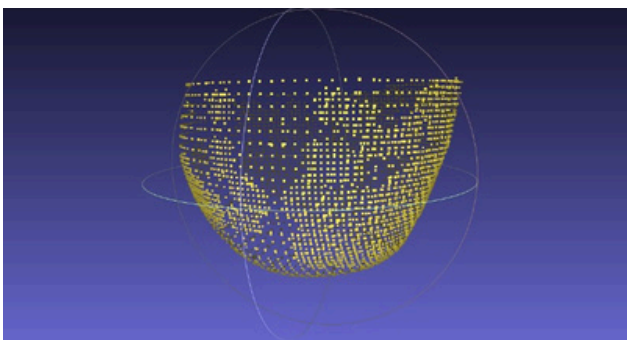
- The .xyz file, containing coordinates derived from the VL6180X sensor and motor positions, is imported into MeshLab.
- The point cloud is rendered in 3D space, allowing intuitive interaction such as rotating, zooming, and panning to inspect the scanned object.

MeshLab also provides advanced features including:

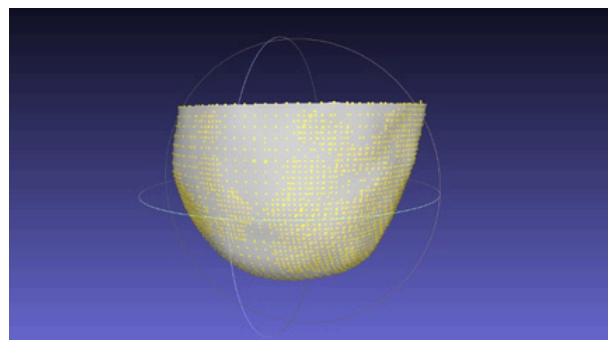
- Mesh reconstruction
- Smoothing and surface cleaning
- Noise reduction and hole filling



Fig e.1 Representation of a simple model on meshlab



point cloud



After remeshing

Modeling

The aim of modeling the system is to represent the equations governing its motion using Simulink blocks, enabling clear visualization and better interpretation of the motor's dynamic response.

Electrical Equations:

- $V_a = R_a * i_a + L_a * (di_a/dt) + e_a$
 - V = applied voltage
 - R = winding resistance
 - i = phase current
 - L = winding inductance
 - e = back EMF
- $e_a = -k_e * \omega * \sin(N_r * \theta)$
 - k_e = back EMF constant
 - ω = angular velocity $\rightarrow d\theta/dt$
 - N_r = number of rotor teeth
 - θ = rotor position

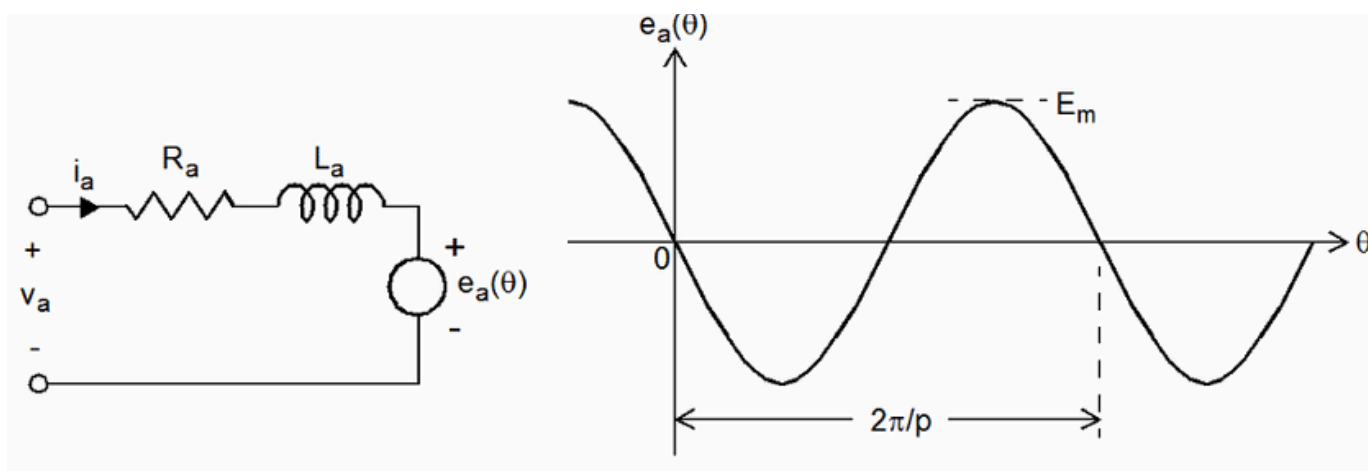


Fig f.1 Stepper motor circuit and phase behaviour

Mechanical Equations:

- $T_e = -k_t \cdot i_a \cdot \sin(N_r \cdot \theta) + k_t \cdot i_b \cdot \cos(N_r \cdot \theta)$
- $T_e - T_L = J \cdot (d^2\theta/dt^2) + B \cdot (d\theta/dt)$
 - T_e = electromagnetic torque
 - k_t = torque constant ($\approx k_e$)
 - J = rotor inertia
 - B = viscous friction coefficient
 - T_L = load torque

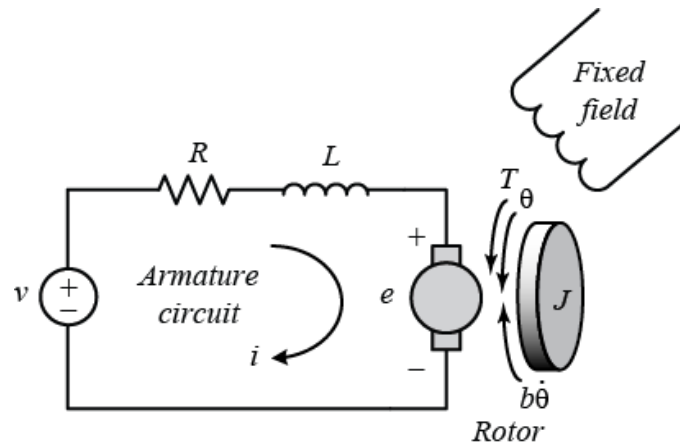


Fig f.2 Stepper motor output torque

Stepper motor in simulink

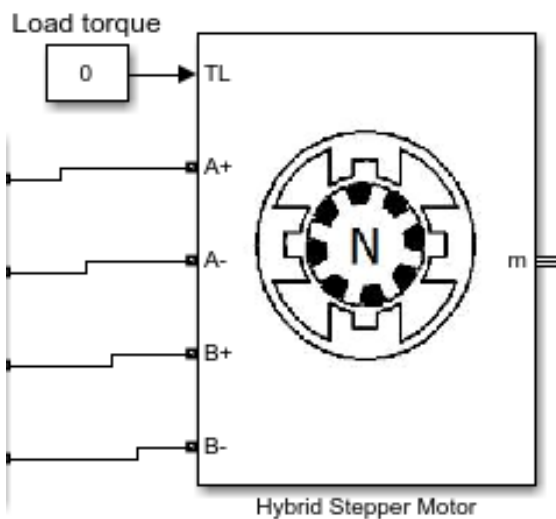


Fig f.3 Stepper motor Block

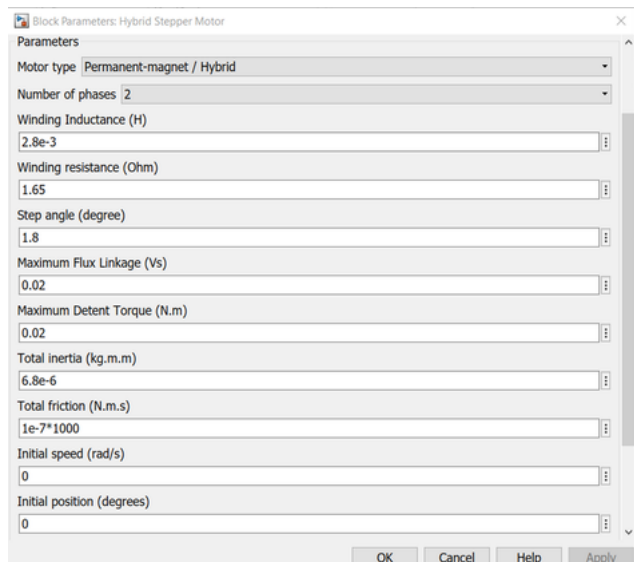


Fig f.4 Adjusting Parameters for nema 23

In Fig f.3 and fig f.4 the stepper motor block is implemented in simulink from the simscape library and the parameters are adjusted to represent the nema 23 motor

Motor driver modeling

The stepper motor driver consist of the following stages:

- Inputs and Control Logic
 - DIR (Direction Input): This signal determines the direction of the stepper motor's rotation. A value of 1 indicates one direction, while -1 reverses it.
 - STEP (Step Pulse Input): This input generates pulses that determine when the motor should take a step forward or backward.
- Direction and Step Control
 - The multiplexer (MUX) block processes the direction (DIR) and step (STEP) signals.
 - A discrete delay ($K / (z-1)$) stores the previous step count, ensuring that steps accumulate correctly.
 - The modulus operation (mod 4) ensures the stepping sequence follows a cyclic pattern, crucial for driving the motor in a synchronized manner.
- Phase Table (TBLA & TBLB)
 - These are lookup tables that map the stepper sequence for two motor windings (A & B).
 - Each winding follows a specific sequence to generate the required magnetic field for stepping.
- Current Reference Generation (I_{ref})
 - The reference current (I_{ref}) determines the magnitude of current supplied to the motor windings.
 - This ensures proper torque output and prevents overheating or excessive power draw.
- Signal Processing and Inversion
 - The computed signals are multiplied with the reference current.
 - The inversion (INVE block) ensures that complementary signals are generated for the full-bridge converter.
 - A low-pass filter (LPF) smooths out high-frequency components before sending signals to the power stage.

- Power Stage (H-Bridge Converters A & B)
 - Converter A and Converter B are H-Bridge circuits controlling the two motor windings.
 - The H-Bridge switches power between positive (V+) and negative (V-) voltages.
 - The NOT gates ensure the correct switching logic.
- Outputs
 - A+ and A- control winding A of the stepper motor.
 - B+ and B- control winding B.
 - The output signals regulate the stepper motor's phase excitation, ensuring proper motion.

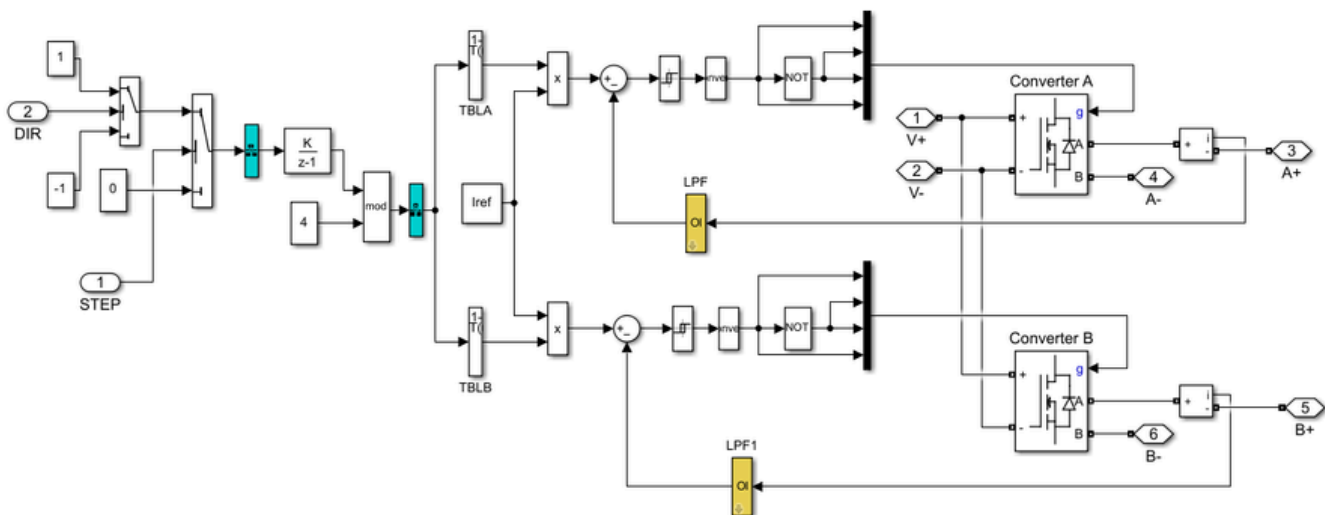


Fig f.5 Block diagram of Motor driver in simulink

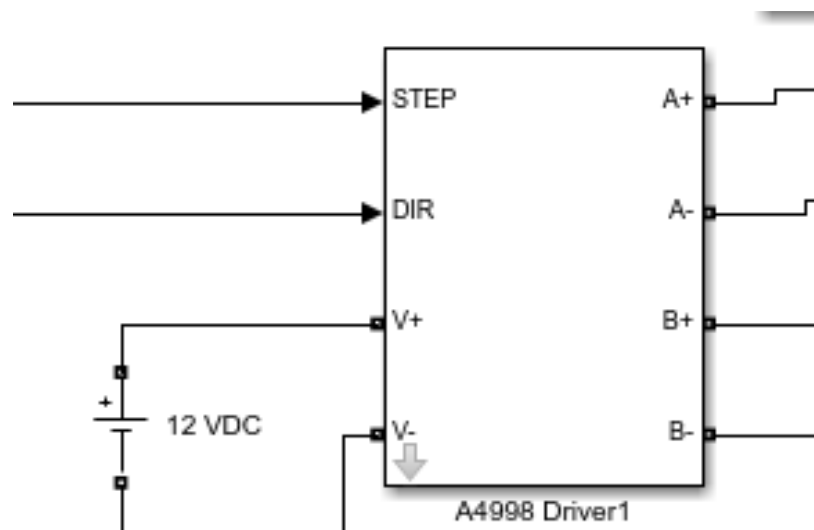


Fig f.6 motor driver subsystem block

Fully integrated model

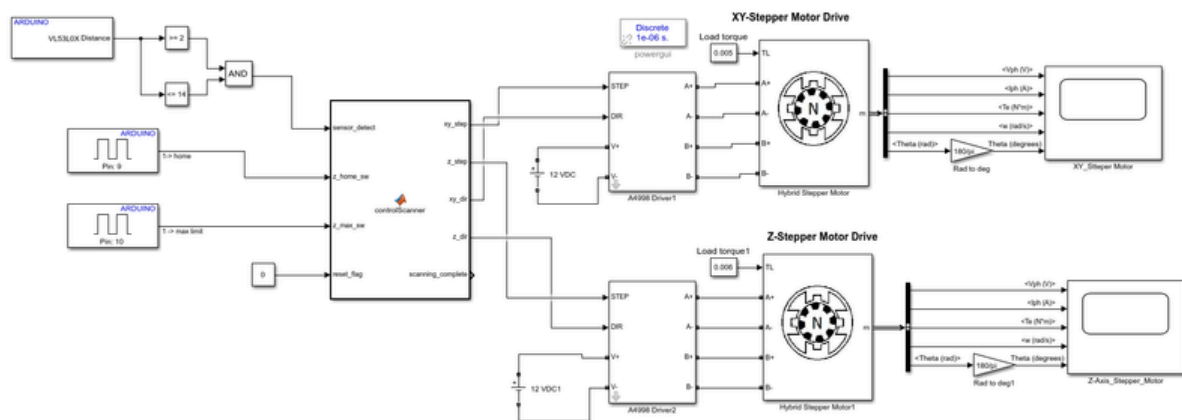


Fig f.7 Complete simulink model

This Simulink model (fig f.7) controls a 2-axis scanning system using Arduino sensor inputs and stepper motors.

- The VL53L0X distance sensor checks object presence, and limit switches control the Z-axis limits.
- A central control block generates step and direction signals for XY and Z axes.
- A4988 drivers drive hybrid stepper motors with modeled mechanical loads.
- Motor performance (voltage, current, torque, speed, and position) is monitored.

Stepper motors response

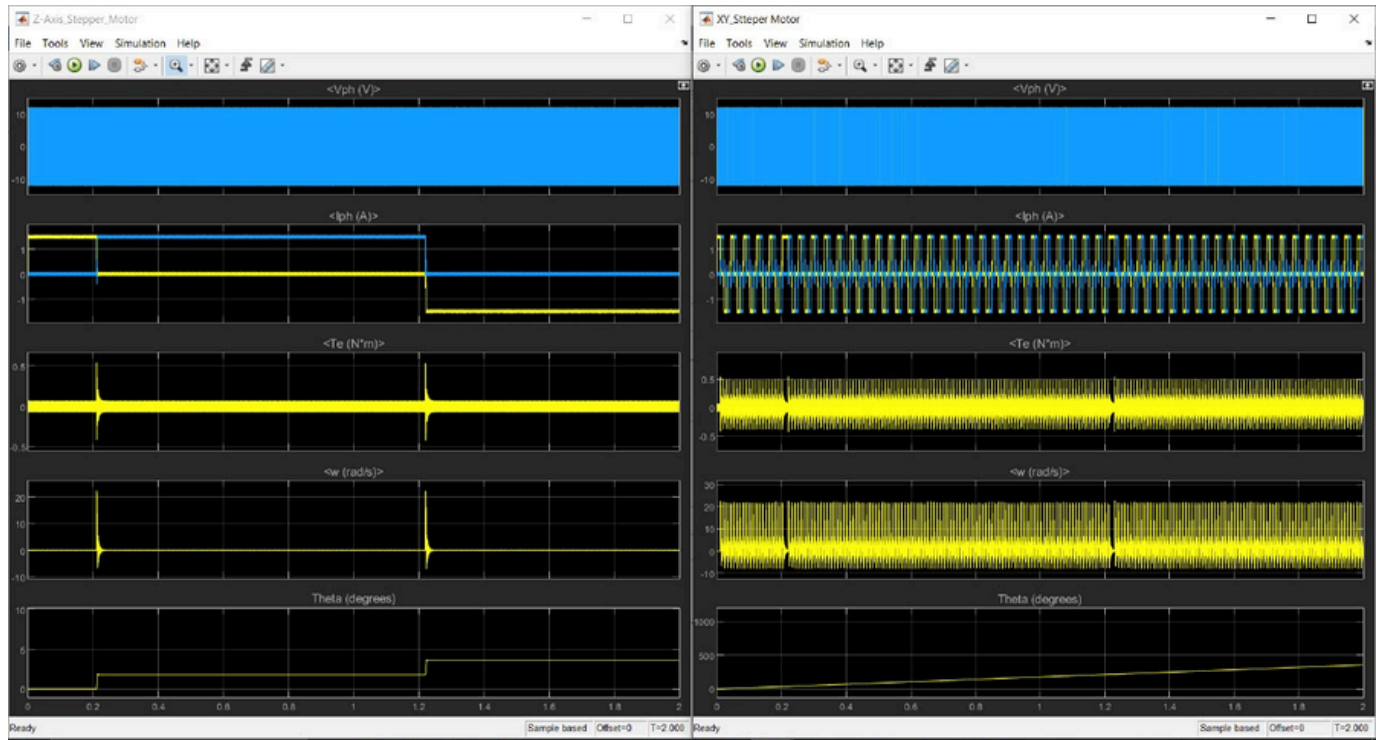


Fig f.8 Motor's response

As shown in Fig. F.8, the XY stepper motor (located on the right side) rotates by a predefined angular step. After completing each angular movement, the Z-axis stepper motor (on the left side) is activated to produce a linear displacement. The Z-axis motor continues moving incrementally until a total linear travel of 2 mm is achieved. This coordinated motion between the two motors allows for precise step-by-step positioning, typically required in scanning or plotting applications.